

**Integrantes:**

- José Proaño
- Cristian Robalino
- Darwin Panchez

Tema: Patrón de Tubería y Filtro

Carrera: Software

NRC: 23305

Fecha: 11/06/2025

1) Definición del patrón de diseño

El patrón Tubería y Filtro organiza el procesamiento de datos como una secuencia de filtros independientes, conectados por tuberías que transportan el flujo de datos. Cada filtro realiza una transformación específica: la entrada entra por su puerto de entrada, se procesa, y se envía al siguiente filtro a través de una tubería

De acuerdo con Sommerville, este estilo encaja bien con sistemas donde los datos fluyen en etapas claramente definidas y los componentes se comunican mediante flujos de datos .

Se suele utilizar en aplicaciones de procesamiento de datos (tanto basadas en lotes [batch] como en transacciones), donde las entradas se procesan en etapas separadas para generar salidas relacionadas.

Principales características:

- **Filtros:** módulos que transforman datos de forma independiente.
- **Tuberías:** canales que conectan filtros, preservando el orden de los datos y sin alterarlos .

El patrón puede aplicarse de forma secuencial o concurrente, con la posibilidad de paralelizar filtros para mejorar el rendimiento

Ventajas

Fácil de entender y soporta reutilización de transformación. El estilo del flujo de trabajo coincide con la estructura de muchos procesos empresariales. La evolución al agregar transformaciones es directa. Puede implementarse como un sistema secuencial o como uno concurrente.

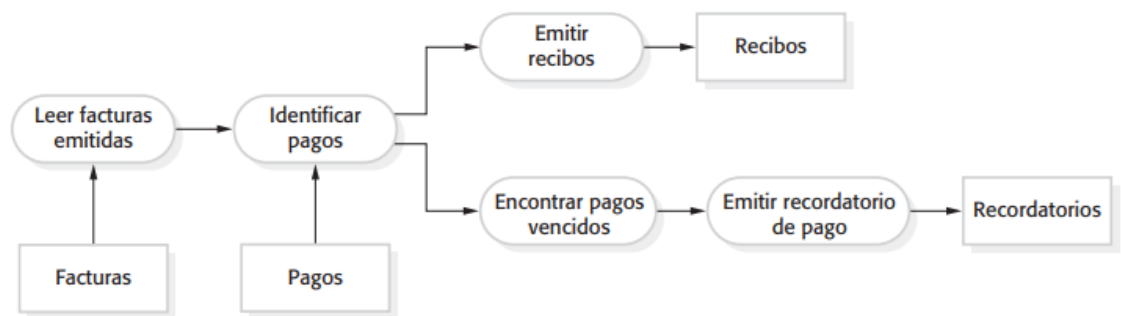
Desventajas



ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

El formato para la transferencia de datos debe acordarse entre las transformaciones que se comunican. Cada transformación debe analizar sus entradas y sintetizar sus salidas al formato acordado. Esto aumenta la carga del sistema, y puede significar que sea imposible reutilizar transformaciones funcionales que usen estructuras de datos incompatibles.

Ejemplo



2) Aplicación en la industria

Este patrón es especialmente útil en sistemas de procesamiento de datos, donde se realizan una serie de transformaciones sobre flujos continuos o archivos en lotes.

Casos de uso comunes

Compiladores:

El flujo clásico de compilación incluye:

- Análisis léxico (scanner)
- Análisis sintáctico (parser)
- Optimización
- Generación de código

Donde cada etapa actúa como un filtro, este diseño modular es típico de compiladores como GCC.

Unix/Linux pipelines:

El uso de `cat file | grep 'foo' | sort | uniq` es un ejemplo directo del patrón en sistemas operativos: cada comando es un filtro, conectado por pipes.



ETL (Extract, Transform, Load) en data warehousing:

- Herramientas como Apache NiFi, Talend, Apache Hop, utilizan pipelines de filtros para extraer (Extract), transformar (Transform) y cargar (Load) datos desde múltiples orígenes hacia destinos consolidados.
- En ingeniería de datos, se encadenan etapas como limpieza, transformación, validación y carga, todo ello mediante componentes modulares dentro de pipelines.

Procesamiento de streaming y Big Data:

Arquitecturas como Apache Flink, Storm o Kafka Streams emplean tuberías y filtros para flujos en tiempo real.

Middleware en aplicaciones web

En frameworks como Express.js o ASP.NET Core, cada middleware (autenticación, logging, validación, gestión de errores) se encadena como filtros en la tubería de procesamiento HTTP

Integración empresarial (Enterprise Integration Patterns):

Sistemas de mensajería aplican filtros para validación, encriptado o duplicación de mensajes.

Procesamiento de transacciones

En arquitecturas cloud (ej. AWS), se usan colas y funciones (Lambda) para:

- autenticar
- validar
- detectar fraudes
- aplicar lógica de negocio

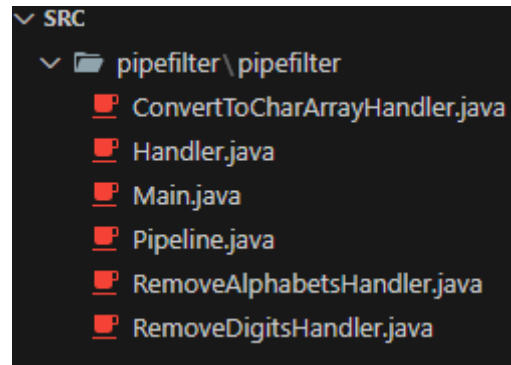
Todo en un flujo de pipelines de mensajería.

3) Ejemplo con código fuente IDE OO (Visual Studio Code con extensión para Java)

Link: <https://onlinegdb.com/6tLmXwYZM>



Estructura del proyecto



1. Handler.java

Interfaz genérica que representa cada etapa (filtro).

```
pipefilter > pipefilter > Handler.java > ...  
1  package pipefilter;  
2  
3  public interface Handler<I, O> {  
4      O process(I input);  
5  }  
6
```

Tiene dos parámetros:

- I (Input): tipo del dato de entrada.
- O (Output): tipo del dato de salida.

En el que recibe un objeto de tipo I, lo procesa y devuelve un objeto de tipo O

2. RemoveAlphabetsHandler.java

Filtro que elimina letras de la cadena de texto.



ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

```
pipefilter > pipefilter > RemoveAlphabetsHandler.java > ...  
1 package pipefilter;  
2  
3 public class RemoveAlphabetsHandler implements Handler<String, String> {  
4     @Override  
5     public String process(String input) {  
6         return input.replaceAll(regex:"[A-Wa-w]", replacement:"");  
7     }  
8 }  
9
```

Elimina todas las letras del abecedario comprendidas entre A y W (mayúsculas) y entre a y w (minúsculas).

Utiliza una expresión regular para hacer el reemplazo “[A-Wa-w]”.

3. RemoveDigitsHandler.java

Filtro que elimina dígitos de la cadena de texto.

```
pipefilter > pipefilter > RemoveDigitsHandler.java > ...  
1 package pipefilter;  
2  
3 public class RemoveDigitsHandler implements Handler<String, String> {  
4     @Override  
5     public String process(String input) {  
6         return input.replaceAll(regex:"\\d", replacement:"");  
7     }  
8 }  
9
```

Utiliza una expresión regular (\\d) que coincide con cualquier dígito del 0 al 9.

Reemplaza cada dígito encontrado por una cadena vacía, es decir, los elimina.

4. ConvertToCharArrayHandler.java

Filtro que convierte la cadena resultante en un arreglo de caracteres.



ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

```
pipefilter > pipefilter > ConvertToCharArrayHandler.java > ...  
1  package pipefilter;  
2  
3  public class ConvertToCharArrayHandler implements Handler<String, char[]> {  
4      @Override  
5      public char[] process(String input) {  
6          return input.toCharArray();  
7      }  
8  }  
9
```

Usa el método incorporado de Java `toCharArray()` para convertir toda la cadena en un arreglo donde cada letra ocupa una posición individual.

5. Pipeline.java

Clase que representa la tubería, encadenando múltiples filtros (Handler).



```
pipefilter > pipefilter > Pipeline.java > ...
1  package pipefilter;
2
3  import java.util.ArrayList;
4  import java.util.Arrays;
5  import java.util.List;
6
7  public class Pipeline<I, O> {
8      private final List<Handler<?, ?>> handlers = new ArrayList<>();
9
10     public <X, Y> Pipeline<I, Y> addHandler(Handler<? super I, ? extends O> first,
11                                             List<Handler<?, ?>> rest) {
12         Pipeline<I, Y> newPipe = new Pipeline<>();
13         newPipe.handlers.addAll(this.handlers);
14         newPipe.handlers.add(first);
15         newPipe.handlers.addAll(rest);
16         return newPipe;
17     }
18
19     public Pipeline<I, O> addHandler(Handler<?, ?> handler) {
20         handlers.add(handler);
21         return this;
22     }
23
24     @SuppressWarnings("unchecked")
25     public O execute(I input) {
26         Object data = input;
27         int step = 1;
28         for (Handler<?, ?> handler : handlers) {
29             data = ((Handler<Object, Object>) handler).process(data);
30             // Mejor impresión para arreglos
31             if (data != null && data.getClass().isArray()) {
32                 if (data instanceof char[]) {
33                     System.out.println("Después del filtro " + step + ": " + Arrays.toString((char[]) data));
34                 } else if (data instanceof int[]) {
35                     System.out.println("Después del filtro " + step + ": " + Arrays.toString((int[]) data));
36                 } else if (data instanceof Object[]) {
37                     System.out.println("Después del filtro " + step + ": " + Arrays.toString((Object[]) data));
38                 } else {
39                     System.out.println("Después del filtro " + step + ": " + data);
40                 }
41             } else {
42                 System.out.println("Después del filtro " + step + ": " + data);
43             }
44             step++;
45         }
46         return (O) data;
47     }
48 }
49
```

Esta clase se encarga de encadenar los filtros (handlers) y ejecutar cada uno secuencialmente, pasando el resultado de uno al siguiente, la clase dispone de :

- Lista que almacena los filtros añadidos a la tubería.
- Cada filtro implementa la interfaz Handler.
- Permite agregar un filtro a la secuencia.
- Devuelve el mismo pipeline para encadenar más filtros.
- Ejecuta toda la secuencia de filtros.
- Toma una entrada y la va procesando paso a paso por cada filtro.



- Se muestra el estado de los datos después de cada filtro

6. Main.java

Programa principal para demostrar la ejecución del pipeline.

```
pipefilter > pipefilter > Main.java > ...
1  package pipefilter;
2
3  import java.util.Arrays;
4
5  public class Main {
6      Run | Debug
7      public static void main(String[] args) {
8          Pipeline<String, char[]> pipeline = new Pipeline<>();
9          pipeline
10             .addHandler(new RemoveAlphabetsHandler())
11             .addHandler(new RemoveDigitsHandler())
12             .addHandler(new ConvertToCharArrayHandler());
13
14             String input = "Abc123XyZ456";
15             char[] result = pipeline.execute(input);
16
17             System.out.println(Arrays.toString(result));
18             // Salida esperada: [X, y, Z]
19         }
20     }
```

- Se crea una instancia de Pipeline.
- Se añaden tres filtros en secuencia:
 1. RemoveAlphabetsHandler: elimina letras de la A a la W (mayúsculas y minúsculas).
 2. RemoveDigitsHandler: elimina todos los dígitos.
 3. ConvertToCharArrayHandler: convierte el resultado en un arreglo de caracteres.
- Se envía una cadena que contiene letras y números.
- Se procesa paso a paso la entrada a través de los filtros.
- Se imprime el resultado final.

Ejecución:



ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

```
Después del filtro 1: 123XyZ456  
Después del filtro 2: XyZ  
Después del filtro 3: [X, y, Z]  
[X, y, Z]
```

En resumen:

- Patrón Tubería y Filtro separa el procesamiento en etapas independientes, conectadas por una tubería que pasa la salida como entrada .
- El código está organizado en archivos por responsabilidad, siguiendo buenas prácticas de diseño OO.
- Ideal para flujos de datos modulares, plantillas de compiladores o procesamiento por lotes.

Bibliografía

- Dhaduk, H., & Dhaduk, H. (2025, January 22). *10 software architecture Patterns You Must know about*. Simform - Product Engineering Company.
<https://www.simform.com/blog/software-architecture-patterns/>
- RobBagby. (n.d.). *Pipes and Filters pattern* - Azure Architecture Center.
Microsoft Learn.
<https://learn.microsoft.com/en-us/azure/architecture/patterns/pipes-and-filters>
- Hasan, S. (2021, December 10). Pipe and Filter Architecture - Syed Hasan - Medium. *Medium*.
<https://syedhasan010.medium.com/pipe-and-filter-architecture-bd7babdb908>
- UNJu Virtual - Plataforma de Educación a Distancia. (n.d.).
<https://virtual.unju.edu.ar/mod/resource/view.php?id=348340&forceview=1>
- GeeksforGeeks. (2024, October 3). Pipe and filter architecture System design.
GeeksforGeeks.



ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

<https://www.geeksforgeeks.org/pipe-and-filter-architecture-system-design/>

- *Pipes and Filters - Enterprise Integration Patterns*. (n.d.). Enterprise Integration Patterns.

<https://www.enterpriseintegrationpatterns.com/patterns/messaging/PipesAndFilters.html>

- Iluwatar. (n.d.). *Pipeline*. Patrones De Diseño Java.

<https://java-design-patterns.com/es/patterns/pipeline/>